FUNKTIONSBESCHREIBUNG • EXPLORATIVES TESTDOKUMENT

4rest

Gamification-basierte Nachhaltigkeitsplattform für Hotels: Gäste und Mitarbeitende erledigen nachhaltige Aufgaben, sammeln Punkte und „pflanzen“ damit Bäume in einem virtuellen Wald.

Frontend: React PWA (Vite) · 3D Wald: Three.js · Backend: Node.js/Express (TypeScript) · Daten: PostgreSQL + Redis

Normale Aktionen: Blaugrün · Abbrechen/Löschen: Rot

Testobjekt (Produktion): `https://luise.bio`

API: `https://luise.bio/api`

Lokal (dev): Frontend `http://localhost:3001` · API `http://localhost:5001`

Inhalt

- 1. Ziel & Testfokus
- 2. Test-Setup (Umgebung, Ports, Accounts, Daten)
- 3. Rollen & Routing (welche Oberfläche sieht wer?)
- 4. Kernfunktionen (User App): Home, Aufgaben, Wald, Rewards, Profil, Statistiken
- 5. Aufgaben-Typen (Video/Quiz/Artikel/Survey/QR/Foto/Visit-Location) – Explorationscharter
- 6. Rewards & Einlösung – Explorationscharter
- 7. Virtueller Wald – Explorationscharter
- 8. Teams & Contests – Explorationscharter
- 9. Admin-/Redaktion-/Rezeption-Oberflächen – Explorationscharter
- 10. Datenschutz (GDPR), Mailbox & Systemverhalten
- 11. Explorative Oracles / Heuristiken
- 12. Bug-Reporting-Checkliste

Dieses Dokument ist absichtlich testorientiert: Es beschreibt Funktionen so, dass man daraus Explorationen/Checks ableiten kann.

1. Ziel & Testfokus

4rest ist eine webbasierte Progressive Web App (PWA), die nachhaltige Handlungen in einem Hotelkontext in spielerische Aufgaben übersetzt. Nutzer erhalten Punkte und visualisieren Fortschritt über einen 3D-Wald. Zusätzlich gibt es administrative Oberflächen für Inhalte (Aufgaben, Rewards, Orte) und Betrieb (Moderation/Tools).

> **Kern-Loop (User):** Aufgabe auswählen → erledigen (je nach Typ) → Punkte erhalten → Fortschritt/Wald aktualisiert → Rewards einlösen.

> **Explorativer Testfokus:** Nicht nur „Happy Paths", sondern bewusst Variationen: Rollen, Datenzustände (leer/teilvoll), Netzwerk/Offline, Berechtigungen, Fehlerpfade, Race Conditions (Doppelklick/Parallelaktionen), Responsiveness und Usability.

2. Test-Setup (Umgebung, Ports, Accounts, Daten)

2.1 Testobjekt **luise.bio**

- **Web-App**: `https://luise.bio` (Login unter `https://luise.bio/login`)
- **REST-API**: `https://luise.bio/api` (z.B. `POST /api/auth/login`)

> **Demo-Accounts (Produktion & Referenz)** – Passwort für alle: `Test123!`
>
E-Mail	Rolle	Oberfläche nach Login
`admin@example.com`	Administrator	Admin-Dashboard (volle Verwaltung)
`rezeption@example.com`	Rezeption	Rezeption-Oberfläche
`redaktion@example.com`	Redaktion	Redaktions-Oberfläche (Aufgaben/Rewards/Orte)
`demo@example.com`	Gast	Normale App (Home, Tasks, Wald, Rewards, ...)

Die Konten werden per Migration `database/migrations/061_luise_demo_accounts.sql` angelegt bzw. synchronisiert (Passwort, Rolle, E-Mail verifiziert). Nach Deployment sind alle vier Logins nutzbar.

2.2 Lokale Entwicklung (optional)

- **Frontend**: `http://localhost:3001`
- **Backend**: `http://localhost:5001`

- Proxy: `/api/*` → Backend.

> Lokal: Demo-User per `database/seed_all.sh` (API-Register + DB) oder nach `npm run migrate` dieselbe Migration 061.

2.3 Datenzustände, die man gezielt herstellen sollte

Leerer Zustand

- Keine Tasks / keine Rewards
- Keine Teams / keine Contests
- Mailbox leer

Oracle: Leere Zustände sollten hilfreich sein (EmptyState, klare CTAs, keine Fehlermeldungen).

Realistischer „voller" Zustand

- Mind. 2–3 Tasks je Typ
- Mind. 5–10 Rewards, unterschiedliche Kosten/Typen
- Teams & Contests aktiv + abgelaufen
- Mind. 1 Foto-Task in „pending" (Validierung)

Oracle: Performance, Filter/Sortierung, Pagination/Tabellenlayout.

3. Rollen & Routing (welche Oberfläche sieht wer?)

Das Frontend routet je nach Rolle auf unterschiedliche „Root"-Oberflächen:

Rolle	Start-Route
administrator	Root zeigt AdminPage (kein normaler Layout-Router)
rezeption	Root zeigt RezeptionPage
redaktion	Root zeigt RedaktionPage
alle anderen	Layout mit: `/` (Home), `/tasks`, `/forest`, `/rewards`, `/statistics`, `/profile`

> Für exploratives Testen ist das wichtig: „Admin" sieht viele User-Flows gar nicht, sondern nur Admin. Daher immer in mehreren Rollen testen.

4. Kernfunktionen (User App)

Home (`/`)

- Übersicht/Einführung, ggf. Kurzstatistiken (Punkte, Bäume, CO_2).
- Navigation zu Tasks/Wald/Rewards.
- Sprache (i18n) und Session-Status (eingeloggt).

Exploration: leere Daten vs. Daten vorhanden; mobile Layout; schnelle Navigation; Reload/Back-Button.

Aufgaben (`/tasks`)

- Liste + Filter (Typ, Completion-Status, „recent“, online, prerequisites).
- Task-Detail je Typ (Video/Quiz/QR/Foto/Visit-Location/Survey/Artikel).
- Verifizierungsstatus (z. B. Foto „pending“).

Exploration: Filterkombinationen, Sortierung (unfertig → pending → fertig), Locks (is_available).

Wald (`/forest`)

- 3D-Canvas + Stats (Punkte, Bäume, Aufgaben, CO_2).
- Aktualisierung über WebSocket Events (task/update) + refreshMe.
- Performance-Settings (dpr, shadows, min performance).

Exploration: Low-End Device, Resize/Rotation, GPU/CPU, Back/Forward, Ladezustände.

Rewards (`/rewards`)

- Liste + Filter (Typ, online rewards), Map-Ansicht für redeemable Rewards.
- Einlösung: Punkte prüfen (User vs Team), Warnung bei Team-Punkten, QR Token/Display.
- Scanner-Flows: PointsBonusScanner, RewardScanner (je nach Rolle/Featureflag).

Exploration: nicht genug Punkte; Team vs User; Double-click Redeem; Token status pending/used/expired.

5. Aufgaben-Typen – Explorationscharter

Die App unterstützt mehrere Aufgabentypen. Für exploratives Testen ist pro Typ ein Set von Charters sinnvoll.

Typ	Was soll funktionieren?	Explorationsideen (Fehler/Edge Cases)
video	Video lädt; Abspielen; Folgefragen/Quiz; Bestehenslogik; Task complete; Punkte steigen; UI aktualisiert.	Netzwerk langsam; Video URL ungültig; Abbruch mitten im Video; Refresh; Doppelklick auf „Abschließen“; Rücksprung in Liste; Fortschritt nach Reload konsistent.
quiz	Fragen/Antworten; Auswahl; Validierung; Bestehen/Fail; UI Feedback; Task complete.	Mehrfachklick; ungültige Antwort-IDs; i18n-Strings fehlen; Zurück-Button; Wiederholbarkeit.
article_quiz	Artikel-Quelle (URL oder Content); Quiz wie oben; Darstellung auf Mobile.	Sehr lange Inhalte; externe Links; Mixed Content; Scroll/Position; Rendering/HTML Sanitization.
survey	Single/Multi/Text/Rating; Required Felder; Submit; Speicherung; Task complete.	Leere Antworten; Sonderzeichen/Emoji; sehr lange Texte; Mehrfachsubmit; Browser zurück.
qr	QR Code anzeigen; Scan via Kamera; Fallback manuell; Offline: speichern/queue; online sync; complete.	Kamera-Rechte verweigert; QR ungültig; mehrfach scan; schneller Wechsel App↔Browser; PWA Offline; iOS Safari Besonderheiten.
photo	Foto aufnehmen/hochladen; Upload; Status „pending“; Admin Validierung; später als completed markieren.	Große Dateien; HEIC; Abbruch Upload; doppelt senden; Serverfehler; Validierung ablehnen; UI nach WebSocket Update.
visit_location	Ort anzeigen; ggf. Karte/Distance; Start Task; Completion wenn am Ziel/Regel erfüllt.	GPS deaktiviert; falsche Permissions; Mock locations; Edge: knapp außerhalb; Flapping GPS; Offline.

Oracles: Bei jeder Task-Completion müssen mindestens drei Dinge konsistent sein: (1) Task in Liste als completed/pending markiert, (2) Punkte im Userprofil/Store aktualisiert, (3) Wald/Stats reflektieren Änderung.

6. Rewards & Einlösung – Explorationscharter

Einlösung (User-Punkte)

- Reward nur einlösbar, wenn genug Punkte.
- Nach Einlösung: Punkte sinken, Einlöse-UI zeigt Token/Code.
- Statuswechsel Token (pending→used/expired) wird angezeigt.

Fehlerpfade: Insufficient points, API Fehler, Timeout, Duplicate request, Reload während Einlösung.

Einlösung (Team-Punkte)

- Wenn Team aktiv: Team totalPoints kann als Budget genutzt werden.
- Warnung wenn Team-Punkte genutzt werden und persönliche Punkte nicht reichen.
- Race: Team totalPoints wird regelmäßig refetched.

Edge: Team verliert Punkte parallel; Einlösung genau auf Grenze; zwei User lösen gleichzeitig ein.

Scanner/QR Flows

- **RewardScanner**: Codes scannen und Status aktualisieren (Admin/Staff abhängig von Rollen/Feature).
- **PointsBonusScanner**: zusätzliche Punkte via QR/Flow (Rollenabhängig).

7. Virtueller Wald – Explorationscharter

Rendering & Interaktion

- Canvas lädt stabil, keine WebGL Crashes.
- Performance akzeptabel (FPS/Responsiveness) auf Laptop + Mobile.
- Resize/Rotation ohne Layout-Breaks.

Datenkonsistenz

- Stats (Punkte/Bäume/CO_2) passen zu Tasks.
- Nach Task completion wird Wald/Stats aktualisiert (Events/Refresh).
- Team-Punkte (wenn vorhanden) korrekt dargestellt.

8. Teams & Contests – Explorationscharter

Teams & Contests erhöhen Motivation über Wettbewerb. In der User-App gibt es u. a. Contest-Ranking-Routen und ggf. Join-Flows über QR.

Bereich	Worauf achten?
Team beitreten/erstellen	QR Join funktioniert; Team ist „active“; Team points werden im Rewards/Wald berücksichtigt; UI aktualisiert ohne Refresh.
Contest beitreten	Registrierung; Zeitraum; Start/Ende; Auswertung; Ranking zeigt korrekte Aggregationen.
Edge Cases	Contest abgelaufen; mehrfach beitreten; Team verlassen; Rechte/rollenspezifische Views.

9. Admin-/Redaktion-/Rezeption-Oberflächen – Explorationscharter

Administrator

- Modi: Task/Reward/Location create & manage, QR-Scanner, SurveyAnalytics, DebugTools, UserManagement, Team/Contest Management, PhotoValidation, PointsBonus.
- Bulk-Operationen (activate/deactivate/delete) mit Selection-State.
- Bestätigungsdialoge: Abbrechen rot, Delete/ConfirmTone danger rot.

Redaktion

- Inhalte verwalten (Edit/Activate/Deactivate) – normale Aktionen in Blaugrün.
- Nur „Löschen“ rot (danger), disabled grau.

Rezeption

- Rezeption-spezifischer Flow (z. B. Teams/Contests/Operations).
- QR/Points Bonus je nach Konfiguration.

Admin Testcharter (Beispiele)

- „Erstelle Task je Typ → prüfe Listing/Filter → prüfe User-App Sichtbarkeit“
- „Aktiviere/Deaktiviere Task → User sieht Locks korrekt“
- „Foto-Validierung: approve/reject → User Task-Status + Punkte“
- „Bulk delete/activate: Selection, disabled states, confirmations“

10. Datenschutz (GDPR), Mailbox & Systemverhalten

GDPR / Consent / Data Export

- Consent-UI erreichbar und verständlich.
- Export liefert vollständige, konsistente Daten (Profil, Tasks, Redemptions).
- Löschung entfernt/anonymisiert Daten; Login danach erwartbar.

Mailbox

- Liste/Detail; „zurück“ Navigation; Löschen rot (danger).
- Leerer Zustand; Pagination/Scroll, Datum/Format.

Systemverhalten: WebSocket Events invalidieren Queries; UI sollte nach Updates konsistent sein (keine „stale“ Punkte).

11. Explorative Oracles / Heuristiken

- **SADT** (Structure/Activity/Data/Timing): Passt Struktur? Aktivität korrekt? Daten korrekt? Timing/Race ok?
- **CRUD+L**: Create/Read/Update/Delete + List/Filter/Sort konsistent.
- **Rollen**: Gleiche Entity (Task/Reward) in unterschiedlichen Rollen prüfen (Sichtbarkeit/Rechte).
- **State**: Refresh, Back/Forward, Reload, Multi-tab – bleibt Zustand konsistent?
- **Offline**: PWA-Offline (QR/Queue), Reconnect, Cache-Invalidation.

- **Input**: Extremwerte (lange Texte, Sonderzeichen), Invalid, Null, leere Sets.
- **Accessibility**: Fokus, Tastatur, Kontrast, Touch targets.
- **Security**: Unautorisierte Zugriffe (Route direkt), CSRF/CORS, Datenleaks.

12. Bug-Reporting-Checkliste

Feld	Beispiel
Rolle / Account	administrator / admin@example.com
Umgebung	dev lokal / prod / Browser + OS
Route / Bereich	/tasks → QRCodeTask
Schritte	1… 2… 3…
Erwartet	Punkte +20, Task completed, Toast success
Ist	Task bleibt pending, Punkte steigen nicht
Reproduzierbarkeit	immer / sporadisch / nur mit Offline
Logs/Screenshots	Console + Network + Screenshot